GlucoDroid iOS Port Plan

GlucoDroid iOS Port — Implementation Plan

Status: Draft — May 2026
Base: JugglucoNG fork (feat-rob branch)
Goal: Ship a native iOS app with the same CGM reading, alerting, TTS, and follower features as the Android fork.

---

1. Executive Summary

JugglucoNG is a heavily native Android application. It combines:

- **Kotlin/Compose UI** — the entire front-end
- **Java services** — Bluetooth scanning, notifications, TTS, alarms
- **C/C++ native layer** — Libre 3 BLE protocol parsing, glucose processing, sensor decryption
- **Room (SQLite) database** — history storage
- **Android-specific APIs** — BLE, NFC, Foreground Service, TextToSpeech

An iOS port cannot be a recompile. It is a partial rewrite around a shared data/protocol core. Estimated effort: **6–10 months** for a single experienced iOS developer with BLE background.

---

2. Architecture Assessment

2.1 What Can Be Shared (C/C++ Core)

Component	Portability	Notes
Libre 3 packet parser	**High**	Pure C++, no Android API calls
Glucose decryption / CRC	**High**	Cryptographic routines are portable

Component	Portability	Notes
Rate-of-change math	**High**	Arithmetic only
BLE packet assembly	**Medium**	Protocol logic yes; BLE API calls no
SQLite schema	**High**	Use raw SQLite or Core Data migration
Alert threshold logic	**High**	Pure Kotlin → translate to Swift

Wrap the C++ core in a thin Swift-callable layer using a **C bridge header** (`bridging-header.h`). The existing CMakeLists can be adapted for Xcode with minor changes.

2.2 What Must Be Rewritten

Android Component	iOS Replacement
BLE (BluetoothGattCallback)	CoreBluetooth (CBCentralManager)
NFC (NfcV / IsoDep)	CoreNFC (NFCTagReaderSession)
Foreground Service	Background Task + BGProcessingTask
TextToSpeech	AVSpeechSynthesizer
Room (ORM)	Core Data **or** raw SQLite via GRDB
Jetpack Compose UI	SwiftUI
WorkManager	BackgroundTasks framework
Notifications	UNUserNotificationCenter
Nightscout follower	URLSession (essentially the same logic)

---

3. iOS Technical Stack

```
GlucoDroid iOS
+-- Swift 6 / SwiftUI 5
+-- CoreBluetooth              BLE sensor communication
+-- CoreNFC                    Libre 2 tap-to-scan
+-- AVFoundation               TTS (AVSpeechSynthesizer)
+-- HealthKit                  Write glucose to Apple Health
+-- BackgroundTasks            Poll / keep-alive when backgrounded
+-- UNUserNotificationCenter   Local alerts
+-- GRDB (SQLite wrapper)      History storage (mirrors Room schema)
+-- C++ bridge                 Libre protocol parser (shared with Android)
```

SwiftUI is the right UI choice — it matches Compose's declarative model conceptually, making the mental translation from Kotlin straightforward.

GRDB (not Core Data) is recommended for the database layer because the schema can be kept close to the existing Room schema, simplifying data migration if users switch devices.

---

4. Key iOS Constraints vs Android

Concern	Android	iOS
BLE background	Foreground Service keeps BLE alive indefinitely	CBCentralManager can reconnect in background; app must hold `bluetooth-central` background mode. State restoration handles termination.
Background execution	Foreground Service (persistent)	BGProcessingTask (15 min slots) + background BLE reconnection. No persistent foreground execution.
NFC	Any NfcV / IsoDep tag	CoreNFC requires explicit user scan session; no passive background NFC. Libre 2 tap-to-scan works; continuous scanning does not.

Concern	Android	iOS
App Review risk	Sideload/F-Droid common	Apple reviews all apps. The Libre 3 BLE protocol is proprietary reverse-engineered; App Store submission may be rejected. **TestFlight + AltStore is the realistic distribution path.**
BLE permissions	Runtime	`NSBluetoothAlwaysUsageDescription` pstring required.

---

5. Phased Implementation Plan

Phase 1 — C++ Core Extraction (Weeks 1–4)

Goal: Isolate and validate the portable C++ layer.

- ☐ Audit `Common/src/main/cpp/` — identify files with zero Android API usage
- ☐ Create a standalone CMake target (`glucocore`) that compiles on macOS/Linux
- ☐ Write unit tests for the glucose parser, decryption, and rate-of-change calculator (can reuse the existing JUnit test logic translated to XCTest)
- ☐ Create a Swift bridging header exposing the C API surface

Output: `libglucocore.a` that links into both Android and iOS targets.

Phase 2 — CoreBluetooth Sensor Driver (Weeks 5–10)

Goal: Receive live Libre 2/3 readings on iPhone.

- ☐ Implement `CBCentralManagerDelegate` scanning loop
- ☐ Port the BLE login handshake from `SuperGattCallback.java` to Swift
- ☐ Port `AccuGattCallback` / Libre 3 patch control characteristic handling
- ☐ Enable `bluetooth-central` background mode in `Info.plist`

- ☐ Implement CBCentralManager state restoration (critical for background reconnect)
- ☐ Validate decrypted glucose values against known test vectors

Libre 3 specifics: The LIBRE3_DATA_SERVICE UUID (`089810cc-...`) and related characteristics are already documented in `SuperGattCallback.java` — these translate directly to CoreBluetooth service/characteristic UUIDs.

Phase 3 — Data Persistence (Weeks 11–13)

Goal: Store history locally, compatible with the Android Room schema.

- ☐ Add GRDB to Xcode project via Swift Package Manager
- ☐ Mirror the glucose history table schema from the existing Room entities
- ☐ Implement sensor identity and session tracking (equivalent to `SensorIdentity.kt`)
- ☐ Write migration logic for users who want to transfer data from Android

Phase 4 — Core Application Logic (Weeks 14–18)

Goal: Glucose display, alerting, TTS, and Nightscout follower.

- ☐ Translate `DisplayDataState.kt` staleness logic to Swift
- ☐ Implement `AVSpeechSynthesizer` TTS with the same voice/speed/separation settings
- ☐ Translate alert threshold logic (`GlucoseAlarms.java`) to Swift
- ☐ Implement `URLSession`-based Nightscout follower (mirrors `NightscoutFollowerManager.kt`)
- ☐ Port the speak-on-stale logic (speaks "Missed readings" when reading is older than 5.5 min)
- ☐ Implement `UNUserNotificationCenter` alerts for high/low/missed

Phase 5 — SwiftUI Application (Weeks 19–26)

Goal: Full working UI on iPhone.

- ☐ Main glucose display (value, trend arrow, graph)
- ☐ Settings screens (voice, alert thresholds, follower URL)
- ☐ Floating overlay equivalent (iOS does not support system overlays; use a widget instead)
- ☐ Lock screen / Dynamic Island live activity (iOS 16+)
- ☐ HealthKit write permission + glucose sample export

Phase 6 — Polish and Distribution (Weeks 27–30)

- ☐ Apple Watch complication (WatchConnectivity + WidgetKit)
- ☐ TestFlight beta distribution
- ☐ AltStore / SideStore manifest for sideloading (fallback if App Store rejected)
- ☐ App Store submission attempt

6. Risk Register

Risk	Likelihood	Impact	Mitigation
App Store rejection (proprietary BLE protocol)	High	High	Plan for AltStore/TestFlight-only distribution from the start; don't depend on App Store
CoreBluetooth background scan killed by iOS	Medium	High	Use state restoration + BGProcessingTask; test on physical device only (simulator has no BLE)
Libre 3 BLE changes breaking protocol	Medium	Medium	Pin to tested firmware versions; implement version detection
NFC passive scanning impossible on iOS	Certain	Low	Inform users that Libre 2 scan requires explicit tap; implement CoreNFC scan session
C++ core has hidden Android JNI dependencies	Medium	Medium	Phase 1 audit catches this early

7. Recommended Approach

Native Swift (not React Native or Flutter) is the right choice because:

1. CoreBluetooth state restoration and background execution require native API depth that cross-platform frameworks paper over.
2. The C++ core can link directly into the Swift app without a JS bridge.
3. SwiftUI composability is close enough to Jetpack Compose that the Kotlin developer can contribute to the iOS codebase with a short ramp-up.
4. HealthKit and Dynamic Island integrations are first-class in native Swift.

Do not attempt React Native or Flutter — both frameworks have significant

gaps in CoreBluetooth background mode support, and the BLE state restoration API is too low-level to surface cleanly through a bridge.

---

8. Effort Estimate

Phase	Weeks	Notes
C++ Core Extraction	4	Can start immediately
CoreBluetooth Driver	6	Needs physical Libre sensor for testing
Data Persistence	3	Straightforward GRDB work
Core Logic	5	Alert/TTS/follower logic
SwiftUI UI	8	Largest variable — depends on design scope
Polish + Distribution	4	Includes Watch support
Total	**~30 weeks**	Solo developer; 20–22 weeks with a dedicated iOS pair

---

9. Immediate Next Steps

1. **Audit the C++ layer** — run a grep for `#include <android/...>` and `JNI` in `Common/src/main/cpp/` to map the Android-specific surface area.
2. **Buy a test iPhone** (or use Simulator for non-BLE work) — CoreBluetooth does not work in the Simulator.
3. **Set up Xcode project scaffold** with the glucocore CMake target linked in.
4. **Prototype the CoreBluetooth scanner** against a real Libre sensor before committing to the full port — BLE background behavior is the highest-risk unknown.